

Chapter 11. Using Optional as a better alternative to null

This chapter covers

- What's wrong with `null` references and why you should avoid them
- From `null` to `Optional`: rewriting your domain model in a null-safe way
- Putting optionals to work: removing null checks from your code
- Different ways to read the value possibly contained in an optional
- Rethinking programming given potentially missing values

Raise your hand if you ever got a `NullPointerException` during your life as a Java developer. Keep it up if this `Exception` is the one you encounter most frequently. Unfortunately, we can't see you at this moment, but we believe that there's a high probability that your hand is raised now. We also guess that you may be thinking something like "Yes, I agree. `NullPointerException`s are a pain for any Java developer, novice, or expert. But there's not much we can do about them, because this is the price we pay to use such a convenient, and maybe unavoidable, construct as `null` references." This feeling is common in the (imperative) programming world; nevertheless, it may not be the whole truth and is more likely a bias with solid historical roots.

British computer scientist Tony Hoare introduced `null` references back in 1965 while designing ALGOL W, one of the first typed programming languages with heap-allocated records, later saying that he did so "simply because it was so easy to implement." Despite his goal "to ensure that all use of references could be absolutely safe, with checking performed automatically by the compiler," he decided to make an exception for `null` references because he thought that they were the most convenient way to model *the absence of a value*. After many years, he regretted this decision, calling it "my billion-dollar mistake." We've all seen the effect. We exam-

Other formats available



Audiobook 



is to determine whether its value is one of
and that we're examining not an object but a
ises that annoying

`NullPointerException`.

In reality, Hoare's statement could underestimate the costs incurred by millions of developers fixing bugs caused by `null` references in the past

50 years. Indeed, the vast majority of the languages^{[1](#)} created in recent decades, including Java, have been built with the same design decision, maybe for reasons of compatibility with older languages or (more probably), as Hoare states, “simply because it was so easy to implement.” We start by showing you a simple example of the problems with `null`.

[1](#)

Notable exceptions include most typed functional languages, such as Haskell and ML. These languages include algebraic data types that allow data types to be expressed succinctly, including explicit specification of whether special values such as `null` are to be included on a type-by-type basis.

11.1. How do you model the absence of a value?

Imagine that you have the following nested object structure for a person who owns a car and has car insurance in the following listing.

Listing 11.1. The Person/Car/Insurance data model

```
public class Person {
    private Car car;
    public Car getCar() { return car; }
}
public class Car {
    private Insurance insurance;
    public Insurance getInsurance() { return insurance; }
}
public class Insurance {
    private String name;
    public String getName() { return name; }
}
```

What’s problematic with the following code?

```
public String getCarInsuranceName(Person person) {
    return person.getCar().getInsurance().getName();
}
```

Other formats available



able, but many people don’t own a car, so what happens if `getCar()` returns `null`? A common unfortunate reference to indicate the absence of a value (here, to indicate the absence of a car). As a consequence, the call to `getInsurance` returns the insurance of a `null` reference, which results in a `NullPointerException` at runtime and stops your program from run-

ning further. But that's not all. What if person was null? What if the method getInsurance returned null too?

11.1.1. Reducing NullPointerExceptions with defensive checking

What can you do to avoid running into an unexpected NullPointerException? Typically, you can add null checks where necessary (and sometimes, in an excess of defensive programming, even where not necessary) and often with different styles. A first attempt to write a method preventing a NullPointerException is shown in the following listing.

Listing 11.2. Null-safe attempt 1: deep doubts

```
public String getCarInsuranceName(Person person) {  
    if (person != null) { 1  
        Car car = person.getCar();  
        if (car != null) { 1  
            Insurance insurance = car.getInsurance();  
            if (insurance != null) { 1  
                return insurance.getName();  
            }  
        }  
    }  
    return "Unknown";  
}
```

- **1 Each null check increases the nesting level of the remaining part of the invocation chain.**

This method performs a null check every time it dereferences a variable, returning the string "Unknown" if any of the variables traversed in this dereferencing chain is a null value. The only exception to this rule is that you're not checking to see whether the name of the insurance company is null because (like any other company) you *know* it must have a name. Note that you can avoid this last check only because of your knowledge of the business domain, but that fact isn't reflected in the Java classes modeling your data.

Other formats available



Audiobook

✕ [Listing 11.2](#) “deep doubts” because it shows a way to check for null, but you doubt that a variable could be null, so you use a deeply nested if block, increasing the indentation level of the code. This technique clearly scales poorly and compromises readability, so maybe you’d like to attempt another solution. Try to avoid this problem by doing something different as shown in the next listing.

Listing 11.3. Null-safe attempt 2: too many exits

```
public String getCarInsuranceName(Person person) {  
    if (person == null) { 1  
        return "Unknown";  
    }  
    Car car = person.getCar();  
    if (car == null) { 1  
        return "Unknown";  
    }  
    Insurance insurance = car.getInsurance();  
    if (insurance == null) { 1  
        return "Unknown";  
    }  
    return insurance.getName();  
}
```

- **1 Each null check adds a further exit point.**

In this second attempt, you try to avoid the deeply nested `if` blocks, adopting a different strategy: every time you meet a `null` variable, you return the string `"Unknown"`. Nevertheless, this solution is also far from ideal; now the method has four distinct exit points, making it hard to maintain. Even worse, the default value to be returned in case of a `null`, the string `"Unknown"`, is repeated in three places—and (we hope) not misspelled! (You may want to extract the repeated string into a constant to prevent this problem, of course.)

Furthermore, the process is error-prone. What if you forget to check whether one property could be `null`? We argue in this chapter that using `null` to represent the absence of a value is the wrong approach. What you need is a better way to model the absence and presence of a value.

11.1.2. Problems with null

To recap our discussion so far, the use of `null` references in Java causes both theoretical and practical problems:

- ***It's a source of error.*** `NullPointerException` is by far the most

Other formats available



erases readability by making it necessary to
checks that are often deeply nested.
it have any semantic meaning, and in partic-

ular, it represents the wrong way to model the absence of a value in a statically typed language.

- ***It breaks Java philosophy.*** Java always hides pointers from developers except in one case: the `null` pointer.

- ***It creates a hole in the type system.*** `null` carries no type or other information, so it can be assigned to any reference type. This situation is a problem because when `null` is propagated to another part of the system, you have no idea what that `null` was initially supposed to be.

To provide some context for other solutions, in the next section we briefly look at what other programming languages have to offer.

11.1.3. What are the alternatives to null in other languages?

In recent years, languages such as Groovy worked around this problem by introducing a *safe navigation operator*, represented by `?.`, to safely navigate potentially `null` values. To understand how this process works, consider the following Groovy code, which retrieves the name of the insurance company used by a given person to insure a car:

```
def carInsuranceName = person?.car?.insurance?.name
```

What this statement does should be clear. A person may not have a car, and you tend to model this possibility by assigning a `null` to the car reference of the `Person` object. Similarly, a car may not be insured. The Groovy safe navigation operator allows you to safely navigate these potentially `null` references without throwing a `NullPointerException` by propagating the `null` reference through the invocations chain, returning a `null` in the event that any value in the chain is a `null`.

A similar feature was proposed and then discarded for Java 7. Somehow, though, we don't seem to miss a safe navigation operator in Java. The first temptation of all Java developers when confronted with a `NullPointerException` is to fix it quickly by adding an `if` statement, checking that a value isn't `null` before invoking a method on it. If you solve this problem in this way, without wondering whether it's correct for your algorithm or your data model to present a `null` value in that specific situation, you're not fixing a bug but hiding it, making its discovery and remedy far more difficult for whoever will be called to work on it next time (likely you in the next week or month). You're sweeping the dirt under the carpet. Groovy's null-safe dereferencing operator is only a bigger

making this mistake without worrying too

Other formats available



Audiobook



such as Haskell and Scala, take a different view. Haskell includes a `Maybe` type, which essentially encapsulates an optional value. A value of type `Maybe` can contain a value of a given type or nothing. Haskell no concept of a `null` reference. Scala has a similar

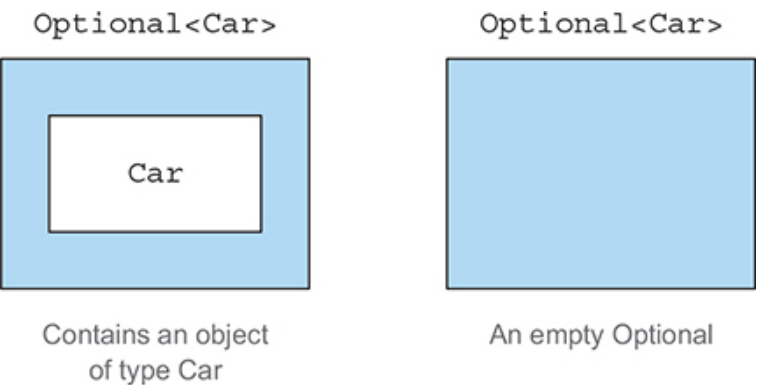
construct called `Option [T]` to encapsulate the presence or absence of a value of type `T`, which we discuss in [chapter 20](#). Then you have to explicitly check whether a value is present or not using operations available on the `Option` type, which enforces the idea of “null checking.” You can no longer forget to check for `null`—because checking is enforced by the type system.

Okay, we’ve diverged a bit, and all this sounds fairly abstract. You may wonder about Java 8. Java 8 takes inspiration from this idea of an optional value by introducing a new class called `java.util.Optional<T>`! In this chapter, we show the advantages of using this class to model potentially absent values instead of assigning a `null` reference to them. We also clarify how this migration from `null`s to `Optionals` requires you to rethink the way you deal with optional values in your domain model. Finally, we explore the features of this new `Optional` class and provide a few practical examples showing how to use it effectively. Ultimately, you learn how to design better APIs in which users can tell whether to expect an optional value by reading the signature of a method.

11.2. Introducing the Optional class

Java 8 introduces a new class called `java.util.Optional<T>` that’s inspired by Haskell and Scala. The class encapsulates an optional value. If you know that a person may not have a car, for example, the `car` variable inside the `Person` class shouldn’t be declared type `Car` and assigned to a `null` reference when the person doesn’t own a car; instead, it should be type `Optional<Car>`, as illustrated in [figure 11.1](#).

Figure 11.1. An optional Car



Other formats available

 Audiobook 

`Optional` class wraps it. Conversely, the absence of a value is represented by an empty optional returned by the `Optional.empty()` static factory method. You may wonder about the difference between a `null` reference and `Optional.empty()`. Semantically, they could be seen as the same thing, but in practice, the difference is

huge. Trying to dereference a null invariably causes a `NullPointerException`, whereas `Optional.empty()` is a valid, workable object of type `Optional` that can be invoked in useful ways. You'll soon see how.

An important, practical semantic difference in using `Optionals` instead of nulls is that in the first case, declaring a variable of type `Optional<Car>` instead of `Car` clearly signals that a missing value is permitted there. Conversely, always using the type `Car` and possibly assigning a null reference to a variable of that type implies that you don't have any help, other than your knowledge of the business model, in understanding whether the null belongs to the valid domain of that given variable.

With this act in mind, you can rework the original model from [listing 11.1](#), using the `Optional` class as shown in the following listing.

Listing 11.4. Redefining the Person/Car/Insurance data model by using `Optional`

```
public class Person {  
    private Optional<Car> car; 1  
    public Optional<Car> getCar() { return car; }  
}  
public class Car {  
    private Optional<Insurance> insurance; 2  
    public Optional<Insurance> getInsurance() { return insurance; }  
}  
public class Insurance {  
    private String name; 3  
    public String getName() { return name; }  
}
```

- **1 A person may not own a car, so you declare this field `Optional`.**
- **2 A car may not be insured, so you declare this field `Optional`.**
- **3 An insurance company must have a name.**

Note how the use of the `Optional` class enriches the semantics of your model. The fact that a person references an `Optional<Car>`, and a car

Other formats available



Audiobook

At the same time, the fact that the name of the insurance company is declared of type `String` instead of `Optional<String>` makes it evident that an insurance company must have a name. This way, you know for

certain whether you'll get a `NullPointerException` when dereferencing the name of an insurance company; you don't have to add a `null` check, because doing so will hide the problem instead of fixing it. An insurance company must have a name, so if you find one without a name, you'll have to work out what's wrong in your data instead of adding a piece of code to cover up this circumstance. Consistently using `Optional` values creates a clear distinction between a missing value that's planned for and a value that's absent only because of a bug in your algorithm or a problem in your data. It's important to note that the intention of the `Optional` class isn't to replace every single `null` reference. Instead, its purpose is to help you design more-comprehensible APIs so that by reading the signature of a method, you can tell whether to expect an optional value. You're forced to actively unwrap an optional to deal with the absence of a value.

11.3. Patterns for adopting Optionals

So far, so good; you've learned how to employ optionals in types to clarify your domain model, and you've seen the advantages of this process over representing missing values with `null` references. How can you use optionals now? More specifically, how can you use a value wrapped in an optional?

11.3.1. Creating Optional objects

The first step before working with `Optional` is learning how to create optional objects! You can create them in several ways.

Empty optional

As mentioned earlier, you can get hold of an empty optional object by using the static factory method `Optional.empty()`:

```
Optional<Car> optCar = Optional.empty();
```

Optional from a non-null value

You can also create an optional from a non-null value with the static fac-

Other formats available



Audiobook



```
= Optional.of(car);
```

If `car` were `null`, a `NullPointerException` would be thrown immediately (rather than getting a latent error when you try to access properties of the car).

Optional from null

Finally, by using the static factory method `Optional.ofNullable`, you can create an `Optional` object that may hold a `null` value:

```
Optional<Car> optCar = Optional.ofNullable(car);
```

If `car` were `null`, the resulting `Optional` object would be empty.

You might imagine that we'll continue by investigating how to get a value out of an optional. A `get` method does precisely this, and we talk more about it later. But `get` raises an exception when the optional is empty, so using it in an ill-disciplined manner effectively re-creates all the maintenance problems caused by using `null`. Instead, we start by looking at ways of using optional values that avoid explicit tests, inspired by similar operations on streams.

11.3.2. Extracting and transforming values from Optionals with map

A common pattern is to extract information from an object. You may want to extract the name from an insurance company, for example. You need to check whether `insurance` is `null` before extracting the name as follows:

```
String name = null;
if(insurance != null){
    name = insurance.getName();
}
```

`Optional` supports a `map` method for this pattern, which works as follows (from here on, we use the model presented in [listing 11.4](#)):

```
Optional<Insurance> optInsurance = Optional.ofNullable(insurance);
Optional<String> name = optInsurance.map(Insurance::getName);
```

This method is conceptually similar to the `map` method of `Stream` you saw in [chapters 4 and 5](#). The `map` operation applies the provided function to

Other formats available



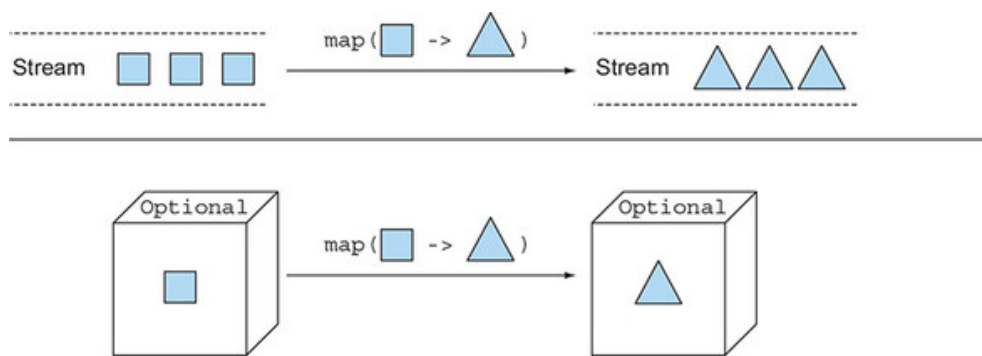
Audiobook



could also think of an `Optional` object as a container of data, containing at most a single element. If the value is `null`, the function passed as argument to `map`

transforms that value. If the `Optional` is empty, nothing happens. [Figure 11.2](#) illustrates this similarity, showing what happens when you pass a function that transforms a square into a triangle to the `map` methods of both a stream of square and an optional of square.

Figure 11.2. Comparing the map methods of Streams and Optionals



This idea looks useful, but how can you use it to rewrite the code in [listing 11.1](#),

```
public String getCarInsuranceName(Person person) {  
    return person.getCar().getInsurance().getName();  
}
```

which chains several method calls, in a safe way?

The answer is to use another method supported by `Optional` called `flatMap`.

11.3.3. Chaining Optional objects with `flatMap`

Because you’ve learned how to use `map`, your first reaction may be to use `map` to rewrite the code as follows:

```
Optional<Person> optPerson = Optional.of(person);  
Optional<String> name =  
    optPerson.map(Person::getCar)  
              .map(Car::getInsurance)  
              .map(Insurance::getName);
```

Unfortunately, this code doesn’t compile. Why? The variable `optPerson` is of type `Optional<Person>`, so it’s perfectly fine to call the `map` method. But `getCar` returns an object of type `Optional<Car>` (as presented in [listing 11.4](#)), which means that the result of the `map` operation is an object

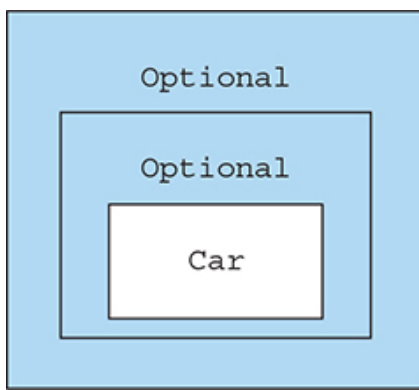
Other formats available

Audiobook



`Optional<Optional<Car>>`. As a result, the call to `getInsurance` in the innermost optional contains as its value another `Optional` that doesn’t support the `getInsurance` method, which is the nested optional structure you’d get.

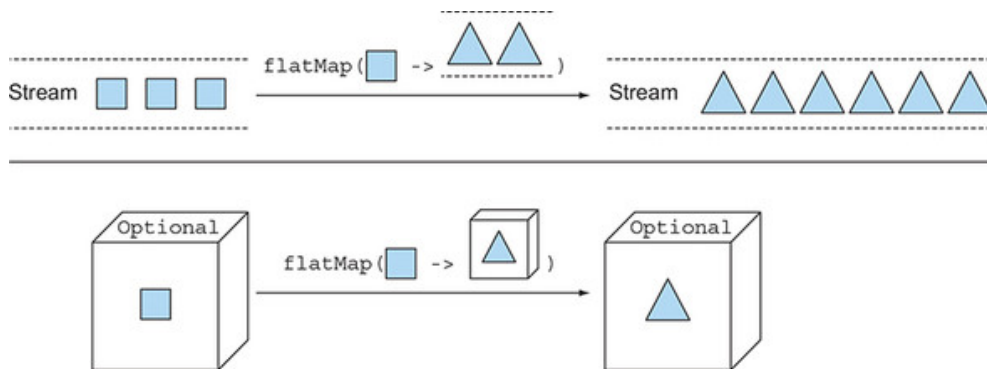
Figure 11.3. A two-level optional



How can you solve this problem? Again, you can look at a pattern you’ve used previously with streams: the `flatMap` method. With streams, the `flatMap` method takes a function as an argument and returns another stream. This function is applied to each element of a stream, resulting in a stream of streams. But `flatMap` has the effect of replacing each generated stream with the contents of that stream. In other words, all the separate streams that are generated by the function get amalgamated or flattened into a single stream. What you want here is something similar, but you want to flatten a two-level optional into one.

As [figure 11.2](#) does for the `map` method, [figure 11.4](#) illustrates the similarities between the `flatMap` methods of the `Stream` and `Optional` classes.

Figure 11.4. Comparing the `flatMap` methods of `Stream` and `Optional`



Here, the function passed to the stream’s `flatMap` method transforms each square into another stream containing two triangles. Then the result of a simple `map` is a stream containing three other streams, each with two triangles, but the `flatMap` method flattens this two-level stream into a single stream containing six triangles in total. In the same way, the function passed to the optional’s `flatMap` method transforms the square con-

Other formats available

Audiobook

into an optional containing a triangle. If the `map` method, the result would be an optional that in turn contains a triangle, but the `flatMap` method flattens this two-level optional into a single optional containing a triangle.

Finding a car’s insurance company name with optionals

Now that you know the theory of the `map` and `flatMap` methods of `Optional`, you're ready to put them into practice. The ugly attempts made in [listings 11.2](#) and [11.3](#) can be rewritten by using the optional-based data model of [listing 11.4](#) as follows.

Listing 11.5. Finding a car's insurance company name with `Optionals`

```
public String getCarInsuranceName(Optional<Person> person) {  
    return person.flatMap(Person::getCar)  
                .flatMap(Car::getInsurance)  
                .map(Insurance::getName)  
                .orElse("Unknown");  
}
```

- **1 A default value if the resulting `Optional` is empty**

Comparing [listing 11.5](#) with the two former attempts shows the advantages of using optionals when dealing with potentially missing values. This time, you can obtain what you want with an easily comprehensible statement instead of increasing the code complexity with conditional branches.

In implementation terms, first note that you modify the signature of the `getCarInsuranceName` method from [listings 11.2](#) and [11.3](#). We explicitly said that there could be a case in which a nonexistent `Person` is passed to this method, such as when that `Person` is retrieved from a database via an identifier, and you want to model the possibility that no `Person` exists in your data for the given identifier. You model this additional requirement by changing the type of the method's argument from `Person` to `Optional<Person>`.

Once again, this approach allows you to make explicit through the type system something that otherwise would remain implicit in your knowledge of the domain model: the first purpose of a language, even a programming language, is communication. Declaring a method to take an optional as an argument or to return an optional as a result documents to your colleagues—and all future users of your method—that it can take an empty value or give an empty value as a result.

Other formats available

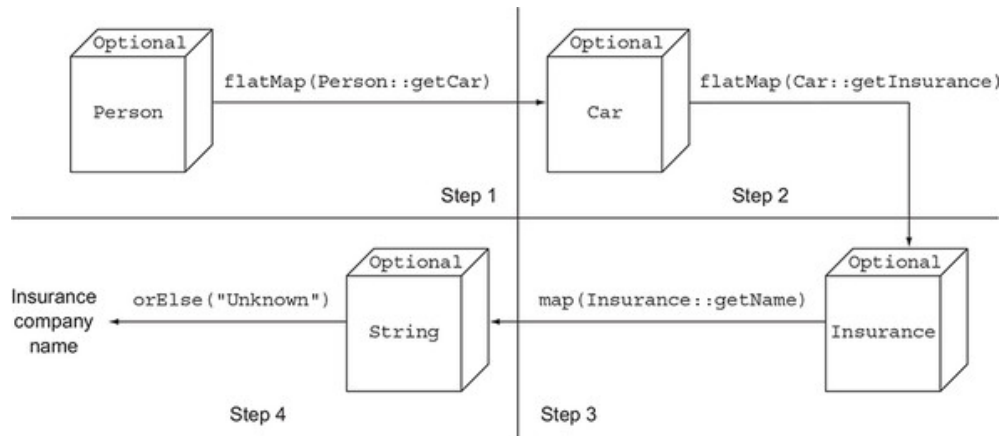


Dereferencing chain using optionals

`Optional<Person>`, the `Car` from the `Person`, the

`Insurance` from the `Car`, and the `String` containing the insurance company name from the `Insurance` are dereferenced with a combination of the `map` and `flatMap` methods introduced earlier in this chapter. [Figure 11.5](#) illustrates this pipeline of operations.

Figure 11.5. The Person/Car/Insurance dereferencing chain using optionals



Here, you begin with the optional that wraps the `Person` and invokes `flatMap(Person::getCar)` on it. As we said, you can logically think of this invocation as something that happens in two steps. In step 1, a `Function` is applied to the `Person` inside the optional to transform it. In this case, the `Function` is expressed with a method reference invoking the method `getCar` on that `Person`. Because that method returns an `Optional<Car>`, the `Person` inside the optional is transformed into an instance of that type, resulting in a two-level optional that's flattened as part of the `flatMap` operation. From a theoretical point of view, you can think of this flattening operation as the operation that combines two nested optionals, resulting in an empty optional if at least one of them is empty. What happens in reality is that if you invoke `flatMap` on an empty optional, nothing is changed, and the empty optional is returned as is. Conversely, if the optional wraps a `Person`, the `Function` passed to the `flatMap` method is applied to that `Person`. Because the value produced by that `Function` application is already an optional, the `flatMap` method can return it as is.

The second step is similar to the first one, transforming the `Optional<Car>` into an `Optional<Insurance>`. Step 3 turns the `Optional<Insurance>` into an `Optional<String>`: because the `Insurance.getName()` method returns a `String`. In this case, a `flatMap` isn't necessary.

At this point the resulting optional will be empty if any of the methods in

Other formats available



an empty optional or otherwise contains the same. How do you read that value? After all, `Optional<String>` that may or may not contain the name of the insurance company. In [listing 11.5](#), we used another method called `orElse`, which provides a default value in case the optional is empty. Many methods provide default actions or unwrap an optional. In the next section, we look at those methods in detail.

Using optionals in a domain model and why they're not serializable

In [listing 11.4](#), we showed how to use `Optional`s in your domain model to mark with a specific type the values that are allowed to be missing or remain undefined. The designers of the `Optional` class, however, developed it based on different assumptions and with a different use case in mind. In particular, Java language architect Brian Goetz clearly stated that the purpose of `Optional` is to support the optional-return idiom only.

Because the `Optional` class wasn't intended for use as a field type, it doesn't implement the `Serializable` interface. For this reason, using `Optionals` in your domain model could break applications with tools or frameworks that require a serializable model to work. Nevertheless, we believe that we've showed you why using `Optionals` as a proper type in your domain is a good idea, especially when you have to traverse a graph of objects that potentially aren't present. Alternatively, if you need to have a serializable domain model, we suggest that you at least provide a method allowing access to any possibly missing value as an optional, as in the following example:

```
public class Person {  
    private Car car;  
    public Optional<Car> getCarAsOptional() {  
        return Optional.ofNullable(car);  
    }  
}
```

11.3.4. Manipulating a stream of optionals



Other formats available

Audiobook

The `filter(Optional::isPresent)` method, introduced in Java 9, allows you to filter a `Stream` of `Optional`s, returning only those `Optional`s that are nonempty. This method can be used to filter a `Stream` of `Optional`s to a `Stream` containing only that value or an empty `Optional` to an equally empty `Stream`. This technique can be particularly convenient in a common case: when you have a `Stream` of `Optional` and need to transform it into another `Stream` containing only the values present in the nonempty `Optional` of the original `Stream`. In

this section, we demonstrate with another practical example why you could find yourself having to deal with a `Stream of Optional` and how to perform this operation.

The example in [listing 11.6](#) uses the `Person/Car/Insurance` domain model defined in [listing 11.4](#). Suppose that you're required to implement a method that's passed with a `List<Person>` and that should return a `Set<String>` containing all the distinct names of the insurance companies used by the people in that list who own a car.

Listing 11.6. Finding distinct insurance company names used by a list of persons

```
public Set<String> getCarInsuranceNames(List<Person> persons) {
    return persons.stream()
        .map(Person::getCar)
        .map(optCar -> optCar.flatMap(Car::getInsurance))
        .map(optIns -> optIns.map(Insurance::getName))
        .flatMap(Optional::stream)
        .collect(toSet());
}
```

- **1 Convert the list of persons into a `Stream of Optional<Car>` with the cars eventually owned by them.**
- **2 `FlatMap` each `Optional<Car>` into the corresponding `Optional<Insurance>`.**
- **3 Map each `Optional<Insurance>` into the `Optional<String>` containing the corresponding name.**
- **4 Transform the `Stream<Optional<String>>` into a `Stream<String>` containing only the present names.**
- **5 Collect the result Strings into a Set to obtain only the distinct values.**

Often, manipulating the elements of a `Stream` results in a long chain of transformations, filters, and other operations, but this case has an additional complication because each element is also wrapped into an `Optional`. Remember that you modeled the fact that a person may not have a car by making its `getCar()` method return an `Optional<Car>` in-

Other formats available



Audiobook



the first map transformation, you obtain a `Stream of Optional<Car>`. At this point, the two subsequent maps allow you to transform each `Optional<Car>` into an `Optional<Insurance>` and then each `Optional<Insurance>` into an `Optional<String>` as you did in [listing](#)

[11.5](#) for a single element instead of a `Stream`.

At the end of these three transformations, you obtain a `Stream<Optional<String>>` in which some of these `Optionals` may be empty because a person doesn't own a car or because the car isn't insured. The use of `Optionals` allows you to perform these operations in a completely null-safe way even in case of missing values, but now you have the problem of getting rid of the empty `Optionals` and unwrapping the values contained in the remaining ones before collecting the results into a `Set`. You could have obtained this result with a `filter` followed by a `map`, of course, as follows:

```
Stream<Optional<String>> stream = ...
Set<String> result = stream.filter(Optional::isPresent)
                        .map(Optional::get)
                        .collect(toSet());
```

As anticipated in [listing 11.6](#), however, it's possible to achieve the same result in a single operation instead of two by using the `stream()` method of the `Optional` class. Indeed, this method transforms each `Optional` into a `Stream` with zero or one elements, depending on whether the transformed `Optional` is empty. For this reason, a reference to that method can be seen as a function from a single element of the `Stream` to another `Stream` and then passed to the `flatMap` method invoked on the original `Stream`. As you've already learned, in this way each element is converted to a `Stream` and then the two-level `Stream` of `Streams` is flattened into a single-level one. This trick allows you to unwrap the `Optionals` containing a value and skip the empty ones in only one step.

11.3.5. Default actions and unwrapping an `Optional`

In [section 11.3.3](#), you decided to read an `Optional` value using the `orElse` method, which allows you to also provide a default value that will be returned in the case of an empty optional. The `Optional` class provides several instance methods to read the value contained by an `Optional` instance:

- `get()` is the simplest but also the least safe of these methods. It returns the wrapped value if one is present and throws a `NoSuchElementException` otherwise. For this reason, using this method is a bad idea unless you're sure that the optional contains a value. In this situation, this method isn't much of an improvement over `getOrNull()`.
- `orElse(T other)` is the method used in [listing 11.5](#), and as we noted there, it allows you to provide a default value when the optional doesn't contain a value.

Other formats available



- `orElseGet(Supplier<? extends T> other)` is the lazy counterpart of the `orElse` method, because the supplier is invoked only if the optional contains no value. You should use this method when the default value is time-consuming to create (to gain efficiency) or you want the supplier to be invoked only if the optional is empty (when using `orElseGet` is vital).
- `or(Supplier<? extends Optional<? extends T>> supplier)` is similar to the former `orElseGet` method, but it doesn't unwrap the value inside the `Optional`, if present. In practice, this method (introduced with Java 9) doesn't perform any action and returns the `Optional` as it is when it contains a value, but lazily provides a different `Optional` when the original one is empty.
- `orElseThrow(Supplier<? extends X> exceptionSupplier)` is similar to the `get` method in that it throws an exception when the optional is empty, but it allows you to choose the type of exception that you want to throw.
- `ifPresent(Consumer<? super T> consumer)` lets you execute the action given as argument if a value is present; otherwise, no action is taken.

Java 9 introduced an additional instance method:

- `ifPresentOrElse(Consumer<? super T> action, Runnable emptyAction)`. This differs from `ifPresent` by taking a `Runnable` that gives an empty-based action to be executed when the `Optional` is empty.

11.3.6. Combining two Optionals

Now suppose that you have a method that, given a `Person` and a `Car`, queries some external services and implements some complex business logic to find the insurance company that offers the cheapest policy for that combination:

```
public Insurance findCheapestInsurance(Person person, Car car) {
    // queries services provided by the different insurance companies
    // compare all those data
    return cheapestCompany;
}
```

Other formats available



Audiobook



develop a null-safe version of this method, taking two optionals as arguments and returning an `Optional<Insurance>` that will be empty if at least one of the values passed in to it is also empty. The `Optional` class also provides an `isPre-`

sent method that returns true if the optional contains a value, so your first attempt could be to implement this method as follows:

```
public Optional<Insurance> nullSafeFindCheapestInsurance(
    Optional<Person> person, Optional<Car> c
) {
    if (person.isPresent() && car.isPresent()) {
        return Optional.of(findCheapestInsurance(person.get(), car.get()));
    } else {
        return Optional.empty();
    }
}
```

This method has the advantage of making clear in its signature that both the Person and the Car values passed to it could be missing and that for this reason, it couldn't return any value. Unfortunately, its implementation resembles too closely the null checks that you'd write if the method took as arguments a Person and a Car, both of which could be null. Is there a better, more idiomatic way to implement this method by using the features of the Optional class? Take a few minutes to go through quiz 11.1, and try to find an elegant solution.

Quiz 11.1: Combining two optionals without unwrapping them

Using a combination of the map and flatMap methods you learned in this section, rewrite the implementation of the former nullSafeFindCheapestInsurance() method in a single statement.

Answer:

You can implement that method in a single statement and without using any conditional constructs like the ternary operator as follows:

```
public Optional<Insurance> nullSafeFindCheapestInsurance(
    Optional<Person> person, Optional<Car> c
) {
    return person.flatMap(p -> car.map(c -> findCheapestInsurance(p, c)));
}
```

Other formats available



✗ In the first optional, so if this optional is passed to it won't be executed, and this in optional. Conversely, if the person is present, flatMap uses it as the input to a Function returning an Optional<Insurance> as required by the flatMap method. The body of this function invokes a map on the second optional, so if it doesn't contain any Car, the Function returns an empty optional, and so does the whole

`nullSafeFindCheapestInsurance` method. Finally, if both the `Person` and the `Car` are present, the lambda expression passed as an argument to the `map` method can safely invoke the original `findCheapestInsurance` method with them.

The analogies between the `Optional` class and the `Stream` interface aren't limited to the `map` and `flatMap` methods. A third method, `filter`, behaves in a similar fashion on both classes, and we explore it next.

11.3.7. Rejecting certain values with filter

Often, you need to call a method on an object to check some property. You may need to check whether the insurance's name is equal to `CambridgeInsurance`, for example. To do so in a safe way, first check whether the reference that points to an `Insurance` object is `null` and then call the `getName` method, as follows:

```
Insurance insurance = ...;
if (insurance != null && "CambridgeInsurance".equals(insurance.getName()))
    System.out.println("ok");
}
```

You can rewrite this pattern by using the `filter` method on an `Optional` object, as follows:

```
Optional<Insurance> optInsurance = ...;
optInsurance.filter(insurance ->
    "CambridgeInsurance".equals(insurance.getName())
    .ifPresent(x -> System.out.println("ok"));
```

The `filter` method takes a predicate as an argument. If a value is present in the `Optional` object, and that value matches the predicate, the `filter` method returns that value; otherwise, it returns an empty `Optional` object. If you remember that you can think of an optional as being a stream containing at most a single element, the behavior of this

Other formats available



Audiobook



optional is already empty, it doesn't have the predicate to the value contained in the optional. If the predicate returns `true`, the optional returns `un-optional`. If the predicate is filtered away, leaving the optional empty.

You can test your understanding of how the `filter` method works by working through quiz 11.2.

Quiz 11.2: Filtering an optional

Supposing that the Person class of your Person/Car/Insurance model also has a method getAge to access the age of the person, modify the getCarInsuranceName method in [listing 11.5](#) by using the signature

```
public String getCarInsuranceName(Optional<Person> person, int minAge)
```

so that the insurance company name is returned *only* if the person has an age greater than or equal to the minAge argument.

Answer:

You can filter the Optional<Person>, to remove any contained person whose age fails to be at least the minAge argument, by encoding this condition in a predicate passed to the filter method as follows:

```
public String getCarInsuranceName(Optional<Person> person, int minAge)
    return person.filter(p -> p.getAge() >= minAge)
                .flatMap(Person::getCar)
                .flatMap(Car::getInsurance)
                .map(Insurance::getName)
                .orElse("Unknown");
}
```

In the next section, we investigate the remaining features of the Optional class and provide more practical examples of various techniques you could use to reimplement the code you write to manage missing values.

[Table 11.1](#) summarizes the methods of the Optional class.

Table 11.1. The methods of the Optional class

Method	Description
	an empty Optional instance
	value is present and matches the given e, returns this Optional; otherwise, returns the empty one
flatMap	If a value is present, returns the Optional resulting from the application of the provided

Other formats available

 Audiobook 

Method	Description
	mapping function to it; otherwise, returns the empty Optional
get	Returns the value wrapped by this Optional if present; otherwise, throws a NoSuchElementException
ifPresent	If a value is present, invokes the specified consumer with the value; otherwise, does nothing
ifPresentOrElse	If a value is present, performs an action with the value as input; otherwise, performs a different action with no input
isPresent	Returns true if a value is present; otherwise, returns false
map	If a value is present, applies the provided mapping function to it
of	Returns an Optional wrapping the given value or throws a NullPointerException if this value is null
ofNullable	Returns an Optional wrapping the given value or the empty Optional if this value is null
or	If the value is present, returns the same Optional; otherwise, returns another Optional produced by the supplying function
orElse	Returns the value if present; otherwise, returns the given default value
orElseGet	Returns the value if present; otherwise, returns the one provided by the given Supplier
	the value if present; otherwise, throws an exception created by the given Supplier
stream	If a value is present, returns a Stream containing only it; otherwise, returns an empty Stream

Other formats available



Audiobook



11.4. Practical examples of using Optional

As you've learned, effective use of the new `Optional` class implies a complete rethink of how you deal with potentially missing values. This rethink involves not only the code you write, but also (and possibly even more important) how you interact with native Java APIs.

Indeed, we believe that many of those APIs would have been written differently if the `Optional` class had been available when they were developed. For backward-compatibility reasons, old Java APIs can't be changed to make proper use of optionals, but all is not lost. You can fix, or at least work around, this issue by adding to your code small utility methods that allow you to benefit from the power of optionals. You see how to do this with a couple of practical examples.

11.4.1. Wrapping a potentially null value in an Optional

An existing Java API almost always returns a `null` to signal that the required value is absent or that the computation to obtain it failed for some reason. The `get` method of a `Map` returns `null` as its value if it contains no mapping for the requested key, for example. But for the reasons we listed earlier, in most cases like this one, you prefer for these methods to return an optional. You can't modify the signature of these methods, but you can easily wrap the value they return with an optional. Continuing with the `Map` example, and supposing that you have a `Map<String, Object>`, accessing the value indexed by key with

```
Object value = map.get("key");
```

returns `null` if there's no value in the map associated with the `String` "key". You can improve such code by wrapping in an optional the value returned by the map. You can either add an ugly `if-then-else` that adds to code complexity, or you can use the method `Optional.ofNullable` that we discussed earlier:

```
Optional<Object> value = Optional.ofNullable(map.get("key"));
```

Other formats available



Audiobook



any time you want to safely transform a value into an Optional.

11.4.2. Exceptions vs. Optional

Throwing an exception is another common alternative in the Java API to returning a `null` when a value can't be provided. A typical example is the

conversion of `String` into an `int` provided by the `Integer.parseInt(String)` static method. In this case, if the `String` doesn't contain a parseable integer, this method throws a `NumberFormatException`. Once again, the net effect is that the code signals an invalid argument if a `String` doesn't represent an integer, the only difference being that this time, you have to check it with a `try/catch` block instead of using an `if` condition to control whether a value isn't `null`.

You could also model the invalid value caused by nonconvertible `Strings` with an empty optional, so you prefer that `parseInt` returns an optional. You can't change the original Java method, but nothing prevents you from implementing a tiny utility method, wrapping it, and returning an optional as desired, as shown in the following listing.

Listing 11.7. Converting a `String` to an `Integer` returning an optional

```
public static Optional<Integer> stringToInt(String s) {  
    try {  
        return Optional.of(Integer.parseInt(s));           1  
    } catch (NumberFormatException e) {  
        return Optional.empty();                           2  
    }  
}
```

- **1 If the `String` can be converted to an `Integer`, return an optional containing it.**
- **2 Otherwise, return an empty optional.**

Our suggestion is to collect several similar methods in a utility class, which you can call `OptionalUtility`. From then on, you'll always be allowed to convert a `String` to an `Optional<Integer>` by using this `OptionalUtility.stringToInt` method. You can forget that you encapsulated the ugly `try/catch` logic in it.

11.4.3. Primitive optionals and why you shouldn't use them

Note that like streams, optionals also have primitive counterparts—

Other formats available



Audiobook

g, and `OptionalDouble`—so the method in `Optional-Int` instead of

Chapter 5, we encouraged the use of primitive

streams (especially when they could contain a huge number of elements) for performance reasons, but because an `Optional` can have at most a single value, that justification doesn't apply here.

We discourage using primitive optionals because they lack the `map`, `flatMap`, and `filter` methods, which (as you saw in [section 11.2](#)) are the most useful methods of the `Optional` class. Moreover, as happens for streams, an optional can't be composed with its primitive counterpart, so if the method of [listing 11.7](#) returned `OptionalInt`, you couldn't pass it as a method reference to the `flatMap` method of another optional.

11.4.4. Putting it all together

In this section, we demonstrate how the methods of the `Optional` class that we've presented so far can be used together in a more compelling use case. Suppose that you have some `Properties` that are passed as configuration arguments to your program. For the purpose of this example and to test the code you'll develop, create some sample `Properties` as follows:

```
Properties props = new Properties();
props.setProperty("a", "5");
props.setProperty("b", "true");
props.setProperty("c", "-3");
```

Also suppose that your program needs to read a value from these `Properties` and interpret it as a duration in seconds. Because a duration has to be a positive (>0) number, you'll want a method with the signature

```
public int readDuration(Properties props, String name)
```

so that when the value of a given property is a `String` representing a positive integer, the method returns that integer, but it returns zero in all other cases. To clarify this requirement, formalize it with a few JUnit assertions:

```
assertEquals(5, readDuration(param, "a"));
assertEquals(0, readDuration(param, "b"));
assertEquals(0, readDuration(param, "c"));
assertEquals(0, readDuration(param, "d"));
```

Other formats available



original requirement: the `readDuration` property "a" because the value of this property in a positive number, and the method returns 0 for "b" because it isn't a number, returns 0 for "c" because it's a number but is negative, and returns 0 for "d" because a property with

that name doesn't exist. Try to implement the method that satisfies this requirement in imperative style, as shown in the next listing.

Listing 11.8. Reading duration from a property imperatively

```
public int readDuration(Properties props, String name) {  
    String value = props.getProperty(name);  
    if (value != null) { 1  
        try {  
            int i = Integer.parseInt(value); 2  
            if (i > 0) { 3  
                return i;  
            }  
        } catch (NumberFormatException nfe) { }  
    }  
    return 0; 4  
}
```

- **1 Make sure that a property exists with the required name.**
- **2 Try to convert the String property to a number.**
- **3 Check whether the resulting number is positive.**
- **4 Return 0 if any of the conditions fails.**

As you might expect, the resulting implementation is convoluted and not readable, presenting multiple nested conditions coded as both `if` statements and a `try/catch` block. Take a few minutes to figure out in quiz 11.3 how you can achieve the same result by using what you've learned in this chapter.

Note the common style in using optionals and streams; both are reminiscent of a database query in which several operations are chained together.

Quiz 11.3: Reading duration from a property by using an Optional

Using the features of the `Optional` class and the utility method of [listing 11.7](#), try to reimplement the imperative method of [listing 11.8](#) with a single fluent statement.

Other formats available



the `Properties.getProperty(String)` method is a `null` when the required property doesn't exist, it's convenient to turn this value into an optional with the `ofNullable` factory method. Then you can convert the `Optional<String>` to an `Optional<Integer>`, passing to its `flatMap` method a reference to the

`OptionalUtility.toStringToInt` method developed in [listing 11.7](#).

Finally, you can easily filter away the negative number. In this way, if any of these operations returns an empty optional, the method returns the `0` that's passed as the default value to the `orElse` method; otherwise, it returns the positive integer contained in the optional. This description is implemented as follows:

```
public int readDuration(Properties props, String name) {  
    return Optional.ofNullable(props.getProperty(name))  
        .flatMap(OptionalUtility::toStringToInt)  
        .filter(i -> i > 0)  
        .orElse(0);  
}
```

Summary

- `null` references were historically introduced in programming languages to signal the absence of a value.
- Java 8 introduced the class `java.util.Optional<T>` to model the presence or absence of a value.
- You can create `Optional` objects with the static factory methods `Optional.empty`, `Optional.of`, and `Optional.ofNullable`.
- The `Optional` class supports many methods—such as `map`, `flatMap`, and `filter`—that are conceptually similar to the methods of a stream.
- Using `Optional` forces you to actively unwrap an optional to deal with the absence of a value; as a result, you protect your code against unintended `null` pointer exceptions.
- Using `Optional` can help you design better APIs in which, by reading the signature of a method, users can tell whether to expect an optional value.

Other formats available



Audiobook

